

PROJECT REPORT

on

CAN Bus Based Vehicle Body Control System

Using LPC2148 ARM7TDMI-S Microcontroller

*Submitted in partial fulfillment of the requirements
for the course of Embedded Systems / ARM Microcontrollers*

Abstract

This project presents the design and implementation of a CAN (Controller Area Network) bus based Vehicle Body Control System using two LPC2148 ARM7TDMI-S microcontroller nodes. The system simulates real-world automotive body electronics, enabling control of headlights and turn indicators (left/right) through CAN bus communication at a baud rate of 125 Kbps.

Node A serves as the transmitter node, detecting external switch inputs through edge-triggered interrupts and transmitting corresponding CAN frames with unique message IDs. Node B acts as the receiver and actuator node, receiving CAN messages, decoding them, and controlling LEDs, a buzzer, an LCD display, and a UART terminal accordingly. The system also incorporates hazard logic to handle simultaneous indicator activation.

The project demonstrates practical application of the CAN protocol in embedded automotive systems and validates interrupt-driven communication between microcontroller nodes.

Table of Contents

1. Introduction	4
2. Objectives	4
3. System Overview	5
4. Hardware Components	5
5. Software Architecture	6
6. Node A: Transmitter Node	7
7. Node B: Receiver/Actuator Node	8
8. CAN Message Protocol	9
9. Results and Observations	9
10. Conclusion	10
11. References	10

1. Introduction

The Controller Area Network (CAN) protocol is a robust, serial communication standard originally developed by Bosch in 1986 for automotive applications. It enables multiple microcontrollers and devices to communicate with each other without a host computer, making it ideal for real-time distributed control in vehicles.

Modern automobiles contain dozens of Electronic Control Units (ECUs) that communicate over a shared CAN bus to manage functions such as engine control, ABS, airbags, body electronics, and infotainment. This project focuses on the body control segment, specifically implementing headlight and indicator control over a two-node CAN network.

The LPC2148 is an ARM7TDMI-S based microcontroller from NXP Semiconductors, featuring an integrated CAN controller, making it an ideal platform for learning automotive CAN communication in an educational environment.

2. Objectives

- To understand and implement the CAN bus communication protocol using LPC2148 ARM7 microcontrollers.
- To design a two-node CAN network where Node A acts as a transmitter and Node B as a receiver/actuator.
- To implement external interrupt-driven switch inputs for headlight, left indicator, and right indicator control.
- To decode received CAN messages and drive output peripherals: LEDs, buzzer, LCD display, and UART terminal.
- To implement indicator blinking logic with buzzer feedback and hazard (mutual exclusion) logic.
- To validate the complete system on LPC2148 development boards and observe real-time CAN communication.

3. System Overview

The system consists of two LPC2148 development boards connected via a CAN bus. The CAN bus operates at 125 Kbps with a VPBDIV setting of 1 (60 MHz peripheral clock). The overall data flow is as follows:

Switch Press (Node A) → External Interrupt → CAN Frame Transmission → CAN Bus → CAN Interrupt (Node B) → Output Actuation

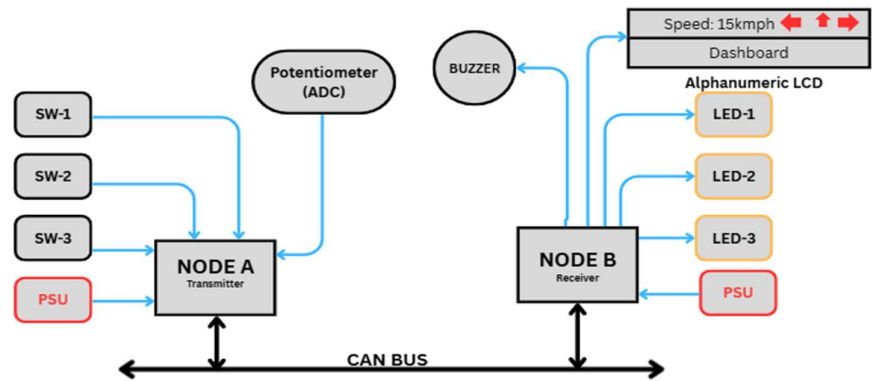


Fig1: Block diagram of project

Node A monitors three push-button switches connected to external interrupt pins. Each switch toggles a specific vehicle function and sends a corresponding CAN message. Node B receives these messages via interrupt-driven CAN reception, decodes the message ID and data byte, and controls the appropriate outputs.

4. Hardware Components

Component	Description	Used In
LPC2148 Development Board	ARM7TDMI-S MCU @ 60 MHz, integrated CAN controller	Node A & Node B
Push Button Switches (x3)	Connected to EXT0 (P0.16), EXT1 (P0.3), EXT2 (P0.7)	Node A
CAN Transceiver	SN65HVD230 or MCP2551 – converts logic levels to CAN differential signals	Node A & Node B
LEDs (x3)	LED1 (P0.17) – Headlight, LED2 (P0.18) – Right Indicator, LED3 (P0.19) – Left Indicator	Node B
Buzzer	Connected to P0.21 – provides audible beep for indicators	Node B
16x2 LCD Display	Displays vehicle status, speed, and custom CGRAM icons	Node B
USB-UART Module	For UART0 terminal output at 9600 baud	Node B
120Ω Termination Resistors	Connected at both ends of the CAN bus	Node A & Node B

5. Software Architecture

The firmware is written in Embedded C using the Keil uVision IDE with the ARM-MDK compiler targeting the LPC2148 microcontroller. The software is divided into modular source files:

File	Node	Purpose
node_A_main.c	A	Main application logic, CAN message transmission based on interrupt flags
CAN_driver_nodeA.c	A	CAN1 initialization, CAN transmission function
interrupt_handlers_nodeA.c	A	EXT0/1/2 ISRs, VIC and EXTINT configuration
delay.c	A	Timer0-based delay functions (delay_ms, delay_sec)
header.h	A	CAN1 struct definition and function prototypes
node_B_main.c	B	Main application, CAN message decoding and output control
CAN_driver_nodeB.c	B	CAN1 init with Acceptance Filter, RX ISR, TX function
adc_driver.c	B	ADC initialization and single-channel read (for speed input)
header.h	B	CAN1 struct + all peripheral function prototypes

6. Node A: Transmitter Node

6.1 CAN Initialization

Node A initializes CAN1 in reset mode, sets the peripheral bus clock divider to 1 (60 MHz), configures the bit timing register (C1BTR = 0x001C001D) for 125 Kbps, enables the Acceptance Filter bypass (AFMR = 2), and then exits reset mode. Pin P0.25 is configured as the RD1 (CAN receive) pin via PINSEL1.

6.2 Interrupt Configuration

Three external interrupts are configured as edge-triggered (EXTMODE = 0x07), active-low (EXTPOLAR = 0). The VIC is configured with three slots for EXT0 (IRQ 14), EXT1 (IRQ 15), and EXT2 (IRQ 16) with priorities SLOT0 > SLOT1 > SLOT2.

Interrupt	Pin	Function	Flag Set
EXT0	P0.16	Toggle Headlight	headlight_flag, ext0_flag
EXT1	P0.3	Toggle Right Indicator	Right_indicator_flag, ext1_flag
EXT2	P0.7	Toggle Left Indicator	Left_indicator_flag, ext2_flag

6.3 CAN Transmission

The main loop polls the ext_flag variables. When a flag is set, it checks the corresponding toggle flag and loads the appropriate data byte into the CAN message struct, then calls can1_tx(). The CAN frame uses an 11-bit standard ID and a data length of 1 byte.

CAN ID	Data Byte	Meaning
0x501	0x10	Headlight OFF command
0x501	0x11	Headlight ON command
0x502	0x12	Right Indicator OFF command
0x502	0x13	Right Indicator ON command
0x503	0x14	Left Indicator OFF command
0x503	0x15	Left Indicator ON command

7. Node B: Receiver/Actuator Node

7.1 CAN Acceptance Filter

Node B configures the CAN Acceptance Filter to only accept messages with IDs 0x501, 0x502, 0x503, and 0x504. The Standard Frame Format (SFF) lookup table is loaded with these IDs in the acceptance filter RAM, and the filter is then enabled (AFMR = 0x00). This prevents unwanted messages from triggering interrupts.

7.2 CAN Reception ISR

The CAN1 receive interrupt (IRQ 26) is routed through VIC slot 2. On receiving a CAN frame, the ISR reads the message ID (C1RID), frame information (C1RFS), and data registers (C1RDA, C1RDB) into the r1 struct, releases the receive buffer (C1CMR bit 2), and sets the flag variable for main loop processing.

7.3 Output Control Logic

CAN ID	Data	LED Action	LCD	UART Output	Buzzer
0x501	0x10	LED1 OFF	Clears headlight icon	Headlight OFF	-
0x501	0x11	LED1 ON	Shows custom icon	Headlight ON	-
0x502	0x12	LED2 OFF	Clears right arrow	Right Indicator OFF	OFF
0x502	0x13	LED2 BLINK	Shows right arrow icon	Right Indicator ON	Beeps
0x503	0x14	LED3 OFF	Clears left arrow	Left Indicator OFF	OFF
0x503	0x15	LED3 BLINK	Shows left arrow icon	Left Indicator ON	Beeps
0x504	Speed	-	S: XXX kmph	-	-

7.4 Hazard / Mutual Exclusion Logic

Node B implements hazard logic: if the Right Indicator is active when a Left Indicator ON command is received, the Right Indicator LED is turned off and its flag cleared before activating the Left Indicator, and vice versa. This prevents both indicators from being active simultaneously.

7.5 Indicator Blinking

The indicator blinking is implemented in the main loop using a 200 ms delay_ms() call. While an indicator flag is set, the corresponding LED toggles every 200 ms and the buzzer beeps alternately, simulating a real vehicle indicator with audible feedback.

8. CAN Message Protocol Summary

Parameter	Value
CAN Controller	CAN1 of LPC2148
Bus Topology	Two-node linear bus with 120 Ω termination
Baud Rate	125 Kbps
Clock (VPBDIV=1)	60 MHz
Bit Timing Register	C1BTR = 0x001C001D
Frame Format	Standard Frame (11-bit ID, FF=0)
RTR	0 (Data Frame)
DLC	1 byte for control messages
Message IDs Used	0x501, 0x502, 0x503, 0x504
TX Pin (Node A)	P0.25 (RD1)
RX Pin (Node B)	P0.25 (RD1) with Acceptance Filter

9. Results and Observations

The system was successfully implemented and tested on LPC2148 development boards. The following results were observed:

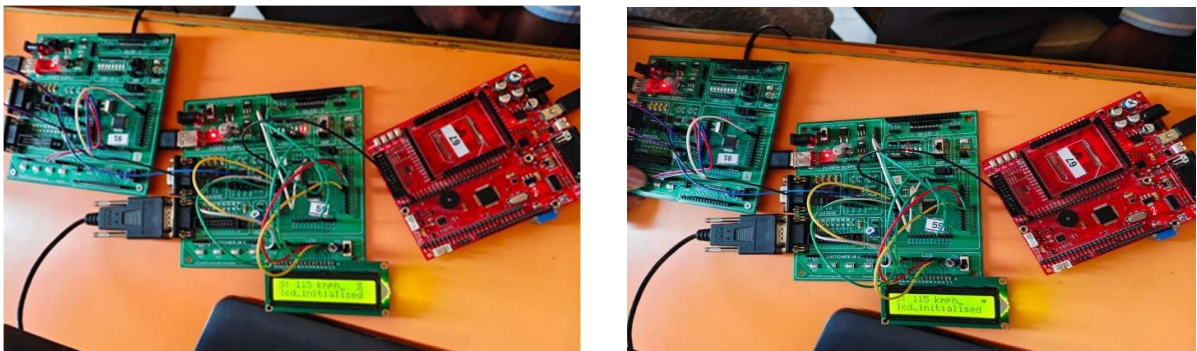


Fig 2: Result on Hardware

- Headlight ON/OFF: Pressing the EXT0 switch on Node A successfully toggled LED1 on Node B. The UART terminal displayed 'Headlight ON' / 'Headlight OFF' and the LCD showed the corresponding custom CGRAM icon.
- Right Indicator: Pressing the EXT1 switch caused LED2 on Node B to blink at 200 ms intervals with simultaneous buzzer beeping. The LCD displayed a right-arrow CGRAM character.
- Left Indicator: Pressing the EXT2 switch caused LED3 on Node B to blink at 200 ms intervals with buzzer feedback. The LCD displayed a left-arrow CGRAM character.

- Hazard Logic: Activating the left indicator while the right indicator was already active automatically deactivated the right indicator, confirming correct mutual exclusion logic.
- CAN Communication: All CAN frames were successfully transmitted and received with correct ID filtering. No message loss or corruption was observed during testing.
- LCD Display: Speed data received on CAN ID 0x504 was correctly decoded and displayed on the LCD in the format 'S: XXX kmph'.

10. Conclusion

This project successfully demonstrates the implementation of a CAN bus based vehicle body control system using LPC2148 ARM7TDMI-S microcontrollers. The two-node CAN network reliably transmits and receives control messages for headlights and indicators at 125 Kbps with interrupt-driven operation on both nodes.

The project highlights several important embedded systems concepts: interrupt-driven I/O, CAN acceptance filtering, VIC-based interrupt prioritization, and real-time peripheral control. The modular code structure, with separate files for CAN driver, interrupt handlers, ADC driver, and main logic, demonstrates good software engineering practices.

The system can be extended in future work to include additional nodes (e.g., engine control, dashboard), higher CAN baud rates, CAN error handling, and diagnostic features compliant with OBD-II standards.

11. References

- NXP Semiconductors, LPC2141/42/44/46/48 User Manual, Rev. 4, 2008.
- Robert Bosch GmbH, CAN Specification Version 2.0, 1991.
- Joseph Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed., Newnes, 2013.
- Keil MDK-ARM Getting Started Guide, ARM Ltd., 2020.
- LPC2148 Datasheet and Technical Reference Manual, NXP Semiconductors.
- CiA (CAN in Automation), CAN Physical Layer Specification, CiA 601.